

Capítulo 10

Física de Cuerpos Rígidos

“A toda acción se opone siempre una reacción igual: las acciones mutuas de dos cuerpos entre sí son siempre iguales y dirigidas en direcciones contrarias.”

—Isaac Newton, *Philosophiæ Naturalis Principia Mathematica*, 1687

Todos los juegos anteriores mueven sus objetos con reglas escritas a mano. La gravedad del Capítulo 4 es una línea de código, la colisión de los Capítulos 3 y 5 es geometría explícita, y la del Capítulo 7 resuelve contra una capa de tiles, no entre cuerpos que a su vez chocan entre ellos. Ese enfoque deja de ser viable en cuanto un juego necesita cuerpos que se apilan, ruedan, se empujan entre sí y se derrumban de forma creíble. Eso es exactamente lo que exige *Throw a Bird*: un ave lanzada por honda contra una torre de piedra y madera que debe mantenerse en pie hasta que algo la golpee, y desmoronarse de forma convincente cuando eso ocurre.

Este capítulo cierra el libro estudiando qué problema resuelve un motor de física real (`pymunk`, un envoltorio de Python sobre `Chipmunk2D`) y cómo `gale.physics` lo envuelve en un vocabulario propio sin ocultar los conceptos que lo sostienen. Como caso de estudio de cada concepto, usa el código real de *Throw a Bird*.

10.1. Contexto histórico: una honda, una torre y un motor de física gratuito

Throw a Bird usa nombres propios, pero su mecánica central, lanzar una criatura con una honda contra una estructura que se derrumba de forma creíble, reconstruye el diseño de *Angry Birds* (Rovio Entertainment, 2009).

Rovio nació en 2003 en Finlandia y sobrevivió durante seis años como estudio subcontratado, desarrollando juegos por encargo sin lograr un éxito propio. Hacia comienzos de 2009, tras 51 títulos que no habían generado ingresos suficientes, la compañía estaba al borde de la quiebra (Gupta & Rood, 2012). *Angry Birds*, lanzado en diciembre de ese año para iOS, fue el intento número 52, y se convirtió en un fenómeno global casi inmediato.

Su mecánica dependía de un motor de física de código abierto escrito en C++ por

el ingeniero Erin Catto: Box2D, el mismo tipo de motor de física de cuerpos rígidos que `gale.physics` envuelve hoy (actualmente, `pymunk` sobre `Chipmunk2D`) y que este capítulo estudia en la práctica. Estaba distribuido bajo una licencia libre que permitió a Rovio simular de forma creíble el desplome de estructuras sin pagar regalías (Plunkett, 2012). Para 2011, el juego acumulaba cientos de millones de descargas y había duplicado los ingresos de la compañía en apenas un año (Cheshire, 2011; Tim et al., 2020). Más allá de las cifras, suele señalarse como el catalizador que popularizó el subgénero de “physics puzzler” en dispositivos táctiles. Demostró que una interacción simple, arrastrar y soltar, combinada con un motor de física convincente, bastaba para sostener sesiones de juego breves, intuitivas y muy repetibles.

10.2. Cuerpos rígidos: estático, dinámico, cinemático

Un motor de física clasifica todo cuerpo rígido en una de tres categorías. Esa clasificación, no la forma ni el tamaño, determina cómo se mueve:

- **Estático:** geometría fija que nunca se mueve, sin importar qué la golpee. El suelo de *Throw a Bird* es exactamente esto: una única caja estática, ancha como todo el nivel.
- **Dinámico:** completamente simulado —gravedad, fuerzas, impulsos, colisiones— todo lo que en este juego puede recibir un golpe: el ave, y cada bloque de piedra, madera o alien de la torre.
- **Cinemático:** se mueve exactamente según una velocidad asignada a mano, sin que la gravedad ni las fuerzas lo afecten, pero empujando a los cuerpos dinámicos que se apoyan sobre él (una plataforma que se desplaza, un ascensor). *Throw a Bird* no usa esta variante: su nivel es completamente estático más una torre dinámica. Aun así, *World* la expone igual, junto a las otras dos, a través de tres constructores paralelos.

La Tabla 10.1 resume estas tres categorías una junto a la otra, con el ejemplo real de *Throw a Bird* que corresponde a cada una, antes de ver el código que las instancia.

Esto se ve, en la práctica, en el código que arma la torre del juego, en `src/entity/Destructible.py`:

```

1 self.body = world.create_dynamic_body(x, y, shape)
2 self.body.angle = angle
3 set_damping(self.body, defn["linear_damping"], defn["
   angular_damping"])
4 self.body.user_data = self

```

y en `src/world/Level.py`, para el suelo:

```

1 self.ground_body = self.world.create_static_body(
2     GROUND_CENTER_X,
3     center_y,

```

Tipo	¿Afectado por gravedad/fuerzas?	¿Cómo se mueve?	Ejemplo en este juego
Estático	No, nunca	No se mueve nunca, sin importar qué la golpee	El suelo (una caja estática, ancha como todo el nivel)
Dinámico	Sí, por completo (gravedad, fuerzas, impulsos, colisiones)	Simulado íntegramente por el motor de física, cuadro a cuadro	El ave, y cada bloque de piedra, madera o alien de la torre
Cinemático	No, la gravedad y las fuerzas no lo afectan	Solo según una velocidad asignada a mano, pero empuja a los cuerpos dinámicos que se le apoyan	Ninguno, ya que <i>Throw a Bird</i> no lo usa, aunque <i>World</i> lo expone igual

Cuadro 10.1: Las tres categorías de cuerpo rígido que reconoce un motor de física, comparadas una junto a la otra.

```

4     BoxShape(GROUND_HALF_WIDTH * 2, GROUND_VISUAL_HEIGHT,
5             friction=1.0),
6 self.ground_body.user_data = "ground"
```

Nótese `user_data`: cada `Body` de `gale.physics` tiene un campo libre donde el juego puede guardar lo que quiera —aquí, o bien el objeto `Destructible` dueño del cuerpo, o el literal `"ground"` para el suelo. Así, cuando el motor de física informe más adelante “estos dos cuerpos chocaron”, el juego puede recuperar de inmediato *a quién* le pasó, sin mantener un mapeo aparte.

10.3. Masa, densidad y forma

Un motor de física no permite fijar la masa de un cuerpo directamente. Solo permite fijar una **densidad** (masa por unidad de área) en cada forma que se le adjunta, y calcula la masa total a partir de esa densidad y del área real de la forma. Esto tiene sentido físico: dos cajas del mismo tamaño hechas de materiales distintos deberían pesar distinto sin cambiar de forma. Pero complica algo tan simple como “quiero que este bloque pese 500”: hay que resolver la densidad que produce esa masa, para el área real de la forma.

Antes de ver las funciones reales, vale la pena escribir la receta en pseudocódigo, para que la traducción de álgebra a código, más abajo, no tenga que hacerse “de un salto”:

```
# Pseudocodigo -- no es codigo Python real
entrada: masa_deseada, forma (con sus medidas en pixeles)
```

```
1. convertir las medidas de la forma de pixeles a metros,
```

dividiendo cada una por PIXELS_PER_METER

2. calcular `area_real` = area de la forma, ya en metros, según su formula geometrica (base x altura para una caja, $\pi * \text{radio}^2$ para un circulo)
3. `densidad` = `masa_deseada` / `area_real`

`salida: densidad`

Throw a Bird resuelve esto con dos funciones en `src/definitions/entity.py`:

```

1 PIXELS_PER_METER = 30.0
2
3 def density_for_box(mass: float, width: float, height: float) ->
  float:
4     area_m2 = (width / PIXELS_PER_METER) * (height /
        PIXELS_PER_METER)
5     return mass / area_m2
6
7 def density_for_circle(mass: float, radius: float) -> float:
8     area_m2 = math.pi * (radius / PIXELS_PER_METER) ** 2
9     return mass / area_m2

```

La idea es despejar: si $\text{densidad} \times \text{área} = \text{masa}$, entonces $\text{densidad} = \text{masa} / \text{área}$, con el área calculada en metros cuadrados (véase la sección de unidades más abajo) a partir del ancho/alto o radio en píxeles. Esto permite que todo el resto del juego, las tablas de datos en ARCHETYPES y BIRD, hable enteramente en términos de una masa deseada (500 para una piedra, 200 para una madera, 2500 para el ave). Ningún otro archivo necesita saber que, por debajo, el motor de física en realidad trabaja con densidad:

```

1 "stone": {
2     "shape": "box",
3     "width": 70,
4     "height": 70,
5     "mass": 500,
6     "friction": 1.0,
7     "restitution": 0.0,
8     ...
9 },

```

y, en `Destructible.__init__`, el punto donde esa masa deseada se convierte en la densidad real que la forma física necesita:

```

1 density = density_for_box(defn["mass"], self.width, self.height)
2 shape = BoxShape(
3     self.width,
4     self.height,
5     density=density,
6     friction=defn["friction"],

```

```

7     restitution=defn["restitution"],
8 )

```

10.4. Unidades: píxeles vs. metros

`pymunk`, el motor detrás de `gale.physics`, espera trabajar en metros, no en píxeles, ya que su solver numérico (heredado de `Chipmunk2D`) está afinado para cuerpos de un tamaño razonable en esa escala (típicamente entre 0.1 y 10 metros). Un cuerpo de “70 metros” (como sería un bloque de 70 píxeles si se pasara directamente) es, para `pymunk`, un objeto absurdamente grande, y la simulación se vuelve numéricamente inestable.

`World` resuelve esto con un factor de conversión, `pixels_per_meter` (30.0 por defecto, el valor que *Throw a Bird* usa sin cambiarlo), que se aplica de forma transparente en ambas direcciones: toda posición que el juego pasa en píxeles se divide por ese factor antes de crear el cuerpo real, y toda posición/velocidad que el juego lee de vuelta se multiplica por él. El juego, y quien lo programa, nunca necesitan pensar en metros; solo la capa de conversión, dentro de `gale.physics`, lo hace.

10.5. Impulso vs. fuerza

Una **fuerza** es continua, pues se aplica cuadro a cuadro mientras algo la sostenga (la gravedad del Capítulo 4 es el ejemplo más simple: se suma a la velocidad vertical en cada `update`). Un **impulso** es, en cambio, un cambio instantáneo de momento: se aplica una sola vez y produce de inmediato un cambio de velocidad completo, sin necesitar fotogramas adicionales para “acumularse”. El lanzamiento de la honda en *Throw a Bird* es un impulso, no una fuerza. El disparo debe sentirse inmediato en el instante en que se suelta el mouse, no como una aceleración que tarda en notarse.

La Figura 10.1 contrasta ambas formas de producir el mismo cambio de velocidad: a la izquierda, una fuerza pequeña sostenida durante muchos cuadros (como la gravedad); a la derecha, un impulso que entrega, en un único instante, un pico equivalente al área completa bajo la curva de la izquierda. Es esa área, no la altura del pico ni la duración de la fuerza, lo que determina el cambio de velocidad resultante.

```

1 def _fling(self) -> None:
2     pull = self.bird.initial_position - self.bird.position
3     scale = FLING_IMPULSE_SCALE * self.bird.mass
4     self.bird.body.apply_impulse(pull.x * scale, pull.y * scale)
5     self.flinging = True
6     self.idle_frames = 0

```

Nótese que el impulso se escala por la propia masa del ave (`self.bird.mass`): como `Body.apply_impulse(ix, iy)` produce un cambio de velocidad de $\Delta v = \text{impulso}/\text{masa}$,

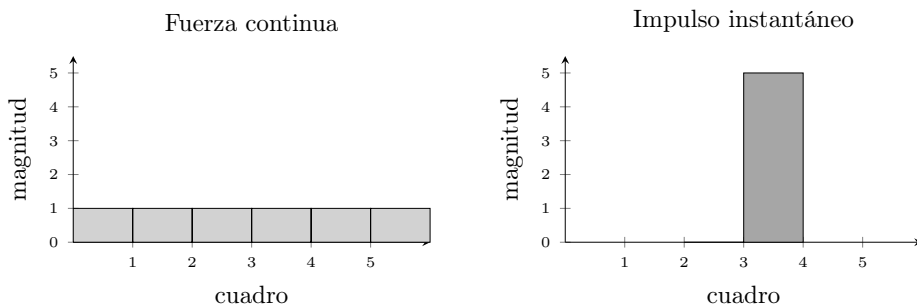


Figura 10.1: Fuerza vs. impulso: la misma área bajo la curva, el mismo cambio total de velocidad, puede lograrse sosteniendo una magnitud pequeña durante muchos cuadros (izquierda) o entregando un único pico equivalente en un solo instante (derecha). El lanzamiento de la honda en *Throw a Bird* usa la segunda forma.

multiplicar el impulso por la masa antes de pasarlo hace que la masa se cancele en el resultado. La velocidad de lanzamiento termina siendo, literalmente, `pull * FLING_IMPULSE_SCALE`, sin importar cuánto pese el ave. `FLING_IMPULSE_SCALE` en sí es puramente un número afinado a mano probando lanzamientos reales, no una constante derivada de ninguna fórmula. No hay una forma de calcular “qué tan fuerte se siente bien” a partir de primeros principios, así que se ajusta por prueba y error hasta que un jalón completo de la honda lanza el ave a una rapidez que se siente correcta.

⚠ Confundir `apply_force` con `apply_impulse`

Es un error fácil de cometer una vez que se entiende la diferencia conceptual pero no se presta atención al nombre del método. Llamar `apply_force` *una sola vez* (como aquí, al soltar la honda) produce un cambio de velocidad casi imperceptible, porque una fuerza necesita sostenerse muchos cuadros para acumular algo notorio, y un solo cuadro a 60 FPS dura apenas $1/60$ de segundo. El error inverso también ocurre: llamar `apply_impulse` *en cada cuadro* (pensando que así se sostiene un empuje continuo) suma un pico completo de velocidad 60 veces por segundo, y el cuerpo sale disparado a una velocidad muy por encima de lo esperado casi de inmediato. La regla práctica: un lanzamiento, un golpe, una explosión son `apply_impulse` de una sola vez; el viento, la gravedad, un empuje sostenido son `apply_force` llamado cuadro a cuadro mientras dure.

10.6. Paso fijo de simulación

El Capítulo 2 establece que el resto del juego escala su movimiento por un Δt variable, el tiempo transcurrido entre cuadros. Un motor de física *no* puede hacer lo mismo con la misma libertad, porque su solver numérico (el algoritmo que resuelve, en cada paso, las fuerzas y colisiones de todos los cuerpos a la vez) es estable solo si el tamaño del paso de tiempo es aproximadamente constante. Un Δt que varíe mucho de un cuadro a otro

(por ejemplo, por un breve tirón del sistema operativo) puede hacer que el solver produzca resultados erráticos: cuerpos que se atraviesan entre sí, o energía que aparece de la nada.

`World` resuelve esto exponiendo `fixed_update()`: un paso real de la simulación, siempre de la misma duración, `fixed_timestep` (por defecto 1/60 segundos):

```
1 def fixed_update(self) -> None:
2     self._space.step(self.fixed_timestep)
```

⚠ Por qué una fuerza “desaparece” si no se reaplica cada `fixed_update`

`pymunk` (heredando el comportamiento de `Chipmunk2D`) borra, al final de cada `space.step()`, todas las fuerzas acumuladas ese paso, de forma automática, sin que el juego tenga que hacer una llamada aparte. Esto es por diseño: se asume que cualquier fuerza continua (viento, gravedad personalizada, un motor que empuja) se va a volver a aplicar en el *siguiente* `fixed_update` si todavía corresponde. Aplicar una fuerza una sola vez, en un único `fixed_update`, y esperar que siga empujando el cuerpo en pasos posteriores es, en la práctica, el mismo error que confundir fuerza con impulso: para un empuje sostenido hay que llamar `apply_force` en *cada* `fixed_update` mientras dure, no una vez y esperar que persista.

Lo único que falta es *cuántas veces por cuadro* llamar a `fixed_update()`, nunca exactamente una, porque el cuadro real casi nunca dura exactamente `fixed_timestep`. Ese es precisamente el acumulador de tiempo del Capítulo 2, Sección 2.4, y `Game` ya lo implementa una vez, a nivel de todo el juego. Por eso *Throw a Bird* no necesita repetirlo: le basta con sobrescribir el método `fixed_update` opcional de `Game` y delegar en el de `World`:

```
1 class ThrowABird(Game):
2     def fixed_update(self) -> None:
3         state = self.state_machine.current
4         fixed_update = getattr(state, "fixed_update", None)
5         if fixed_update is not None:
6             fixed_update()
```

```
1 class PlayState(BaseState):
2     def fixed_update(self) -> None:
3         self.world.fixed_update()
4
5     def update(self, dt: float) -> None:
6         self.level.update(dt)
7         ...
```

`ThrowABird.fixed_update` reenvía la llamada al estado actual de la máquina de estados, ya que solo `PlayState` tiene un mundo físico que avanzar. `VictoryState` no define `fixed_update`, así que `getattr(..., None)` simplemente no hace nada mientras esté activo. `PlayState.fixed_update` llama, a su vez, al `fixed_update` de `World`.

Nótese lo que *ya no* aparece en `PlayState.update`: una llamada a `self.world.update(dt)` habría hecho que `World` corriera *su propio* acumulador interno, por encima del que `Game` ya está corriendo. Serían dos acumuladores independientes tratando de resolver el mismo problema. Por eso la física avanza enteramente a través del acumulador de `Game` y el método de un solo paso de `World`, sin duplicar el mecanismo.

La Figura 10.2 vuelve visible, en un solo diagrama, la cadena completa de cuatro saltos que el código anterior solo muestra en bloques separados: desde el acumulador propio del motor hasta el único `step()` de `pymunk` que efectivamente mueve los cuerpos. Nótese que cada salto es una simple delegación de una llamada a la siguiente: ningún eslabón de la cadena hace nada más que reenviar, hasta llegar a `World`, el único que en verdad avanza la simulación.

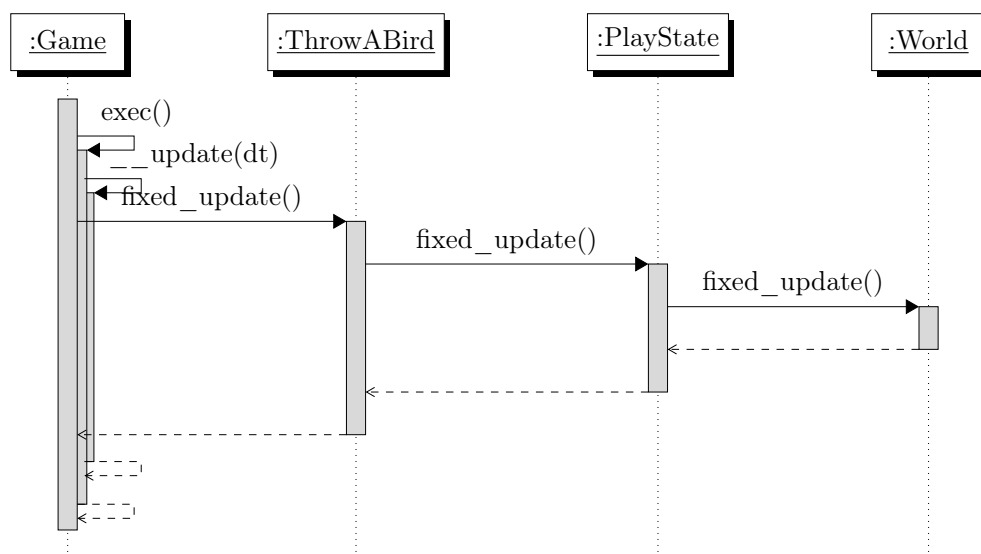


Figura 10.2: Diagrama de secuencia de un paso fijo de simulación en *Throw a Bird*: `Game.exec()` llama a su propio `__update(dt)`, que, tras la lógica del acumulador de tiempo del Capítulo 2, omitida aquí, invoca `fixed_update()` sobre sí mismo; `ThrowABird.fixed_update()` reenvía la llamada al estado actual (`PlayState.fixed_update()`), que a su vez llama a `World.fixed_update()`, el único punto de la cadena que efectivamente avanza el solver de `pymunk`.

10.7. Colisión: callbacks vs. touching_bodies

`World` ofrece dos formas de enterarse de una colisión, para dos necesidades distintas. Para un evento discreto (un impacto que debe hacer daño una sola vez), un callback registrado con `on_collision_begin`:

```

1 self.world.on_collision_begin(self._on_collision)
2
3 def _on_collision(self, body_a, body_b) -> None:
4     self.level.handle_collision(body_a, body_b)

```

que `Level.handle_collision` traduce de vuelta al vocabulario del juego, recuperando el `Destructible` dueño de cada cuerpo desde su `user_data` y decidiendo si corresponde aplicar daño (el bloque no debe dañarse a sí mismo, ni un choque contra el sensor de viento cuenta como impacto físico).

Para una pregunta continua (“¿este cuerpo sigue tocando algo?”), en cambio, no hace falta ningún callback. Basta con consultar `body.touching_bodies` en el momento que se necesite, como hace la zona de viento cada cuadro (véase la última sección de este capítulo).

10.7.1. Daño por velocidad de impacto

El sistema de daño de cada bloque, en `src/entity/Destructible.py`, es la aplicación completa de `on_collision_begin`:

```

1 def on_collision(self, other_body, is_ground: bool) -> None:
2     if self.destroyed:
3         return
4
5     if is_ground:
6         speed = self.body.velocity.length()
7         other_mass = GROUND_EFFECTIVE_MASS
8     else:
9         speed = other_body.velocity.length()
10        other = other_body.user_data
11        other_mass = getattr(other, "mass", GROUND_EFFECTIVE_MASS
12                               )
13
14    if speed <= DAMAGE_SPEED_THRESHOLD:
15        return
16
17    damage = speed * DAMAGE_COEFFICIENT * other_mass / self.mass
18    self.energy -= damage
19
20    if self.energy <= 0:
21        self.destroyed = True

```

El daño es proporcional a la rapidez del impacto y a la masa de lo que golpea, e inversamente proporcional a la propia masa del bloque: un bloque pesado absorbe mejor un golpe que uno liviano, y un golpe rápido duele más que uno lento. Hay, además, un umbral (`DAMAGE_SPEED_THRESHOLD`) por debajo del cual un contacto se considera reposo, no impacto, para que una torre que simplemente se asienta no vaya perdiendo energía cada cuadro. Cada bloque recorre tres niveles de sprite de daño según cuánta energía le queda:

```

1 def current_sprite_name(self) -> str:
2     ratio = max(0.0, self.energy) / self.initial_energy

```

```

3     level = min(3, max(1, math.ceil(3 * ratio))) if ratio > 0
        else 1
4     return self.sprites[level - 1]

```

10.8. Sensores

Una forma marcada como **sensor** detecta solapamiento, dispara `on_collision_begin/on_collision_end` y aparece en `touching_bodies`, sin generar ninguna respuesta física de colisión. Es decir, los cuerpos la atraviesan como si no estuviera, físicamente, mientras el juego decide qué hacer con la información de que se tocaron. *Throw a Bird* usa esto para dos zonas de “viento”, paredes invisibles a los costados del área jugable que devuelven cualquier cosa que se aleje demasiado (el ave en un lanzamiento desviado, o un bloque que salió disparado de la torre), en lugar de dejarla caer para siempre fuera del nivel:

```

1 self.wind_left = self.world.create_static_body(
2     PLAY_AREA_LEFT,
3     center_y,
4     BoxShape(WIND_ZONE_WIDTH, WIND_ZONE_HEIGHT, is_sensor=True),
5 )
6 self.wind_left.user_data = "wind"

```

y, cada cuadro, la zona revisa quién la está tocando y empuja su velocidad de vuelta hacia el área de juego, suavizado con el mismo suavizado exponencial del Capítulo 1 en lugar de un frenazo brusco:

```

1 def _apply_wind(self, dt: float) -> None:
2     factor = 1.0 - math.exp(-WIND_BLEND_RATE * dt)
3     for wind_body, sign in ((self.wind_left, 1), (self.wind_right, -1)):
4         for body in wind_body.touching_bodies:
5             target_vx = sign * WIND_BOUNCE_SPEED
6             vx = body.velocity.x
7             if (sign > 0 and vx < target_vx) or (sign < 0 and vx > target_vx):
8                 body.set_velocity(vx + (target_vx - vx) * factor,
                                   body.velocity.y)

```

Esta es una decisión de diseño concreta, no la única posible. Una primera versión de esta zona aplicaba una fuerza continua mientras hubiera contacto, pero a la velocidad con la que un objeto cruza una zona angosta, esa fuerza no alcanzaba a revertir su movimiento antes de que saliera por el otro lado. Ensachar la zona y frenar activamente la velocidad hacia un objetivo, en lugar de solo empujar, es lo que hace que la zona de viento cumpla su propósito de forma confiable sin importar qué tan rápido llegue algo a ella.

10.9. Construcción de *Throw a Bird*



Figura 10.3: Una partida en curso de *Throw a Bird*.

Construcción completa: un ave lanzada por honda, apuntado por arrastre del mouse, con la posición de aim recalculada cada cuadro para no depender solo de eventos de movimiento del mouse, e impulso aplicado al soltar. A eso se suma una torre de bloques de piedra, madera y aliens con el sistema de daño de la sección anterior, y generación de escombros al destruirse un bloque de madera. También hay detección de reposo (velocidad lineal y angular por debajo de un umbral durante suficientes cuadros seguidos) para devolver el ave a la honda, una cámara que sigue al ave en vuelo y hace zoom hacia afuera según qué tan lejos ha llegado, y las zonas de viento descritas arriba. Un estado de victoria (**VictoryState**) se activa cuando todos los bloques de tipo alien de la torre han sido destruidos.

A diferencia de la máquina de tres estados de Flappy Bird (Figura 4.5), **ThrowABird** usa **StateMachine** con solo dos: **play** (**PlayState**) y **victory** (**VictoryState**). No existe un estado de *game over*, así que mientras la torre siga en pie, el jugador puede seguir lanzando aves indefinidamente. La Figura 10.4 muestra ambos estados y sus dos transiciones reales.

10.10. Ejercicios propuestos

1. Calcular la densidad necesaria para que una caja de 100×50 píxeles tenga una masa objetivo de 300, dado un factor de conversión de 30 píxeles por metro, y verificarlo instanciando el cuerpo con `gale.physics` y leyendo su masa real.
2. Explicar, con un ejemplo numérico, por qué aplicar una fuerza grande durante un único cuadro no produce el mismo resultado que un impulso equivalente, y en qué condiciones ambos convergen al mismo comportamiento.
3. Modificar `DAMAGE_COEFFICIENT` y describir, con números concretos, cómo cambia

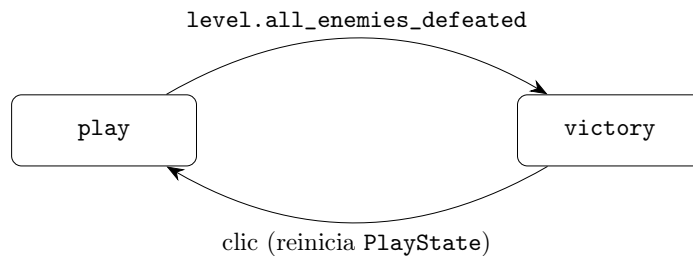


Figura 10.4: Máquina de estados finitos de *Throw a Bird*: la transición `play` $\rightarrow$ `victory` ocurre cuando `PlayState.update` detecta `self.level.all_enemies_defeated` en cada cuadro. La transición `victory` $\rightarrow$ `play` ocurre al hacer clic sobre `VictoryState`, y no es una pausa ni una continuación: `StateMachine.change("play")` instancia un `PlayState` enteramente nuevo, así que el juego reinicia desde cero.

cuántos impactos hacen falta para destruir un bloque de piedra (energía 2000) frente a uno de madera (energía 1000), dado el mismo impacto.

4. Diseñar una zona sensor que aplique un efecto distinto a empujar (por ejemplo, un multiplicador de daño) y discutir qué cambia entre implementarlo como una modificación continua de velocidad (como el viento de este capítulo) o como un efecto de un solo disparo en `on_collision_begin`.